

Rebuilding the SMP Branch from xPack-Development

**Analysis, Architectural Invariants & Step-by-Step
Implementation Runbook**

Rebuilding the SMP Branch from xpack-development

- Comprehensive Architectural Analysis, Step-by-Step Implementation Runbook & GitHub Pull Request Blueprint
- 1. Executive Summary & The GitHub PR Paradigm
- 2. Repository Architecture & Workspace Bootstrap Procedure
- 3. Phase 0: Pre-Migration Subtree Dissolution of Legacy devices
- 4. Phase 1: Single-Core Defect Corrections & Hardening (PR #1 through PR #22)
- 5. Phase 2: Common Multi-Core SMP Kernel Infrastructure & Host Runtime (PR #23 / Step 14)
- 6. Phase 3: Semantic Versioning, Port Releases & Silicon Extensions (PR #24 through PR #27)
- 7. Phase 4: Multi-Core Test Suites, Multi-Arch Platforms & CI Harness (PR #28 & PR #29)
- 8. Phase 5: Documentation Suite, Master Reconciliation & Silicon Wiring (PR #30 through PR #32)
- 9. Operator's Execution Manual: The Step-by-Step Lifting Loop
- 10. Comprehensive Verification Matrix & Acceptance Gates

xpack-development

Unrelated Single-Core Defect Fixes (POSIX dirent.h)

While the resulting `smp` branch in `~/Work2/micro-os-plus/`

branch is rebuilt from scratch, starting directly from clean upstream `origin/xpack-development`

directional Propagation: Commits developed on the new smp

THE NEW REBUILD & PROPAGATION PARADIGM

```

+-----+
|
| UPSTREAM REPOS (xpack-development) <-----+
|
| | Clean Cherry-Picks &
| | git clone & checkout | Focused GitHub PRs
| | v | (PR #1 .. PR #32)
| REBUILD WORKSPACE ($WORK)
|
| +---> Phase 0: Dissolve `devices` subtree
|
| +---> Phase 1: 22 Single-Core Bugfixes & Hardening (PR #1..#22) -----+
| | Gate: 72/72 Uniprocessor tests 100% GREEN
| |
| +---> Phase 2: Common SMP Scheduler & Host native-smp (PR #23) -----+
| | Gate: unifdef zero-diff + 72/72 baseline + native-smp (3/3)
| |
| +---> Phase 3: Semantic Releases & Modern Silicon Ports (PR #24..#27) -----+
| | Gate: Cortex-M33 (MPS2 AN521 2-core) + RP2350 clean
| |
| +---> Phase 4: Multi-Core FPU Stress & 4-Core Platforms (PR #28..#29) -----+
| | Gate: fp-switch + test-smp-all (Native, M33, AArch32/64)
| |
| +---> Phase 5: Documentation Suite & Hardware Wiring (PR #30..#32) -----+
| | Final Parity: 100% Identical to ~/Work2/micro-os-plus/
|
+-----+

```

1.3 Target Architecture: The Five Clean Repositories & Dissolution of **devices**

In the original architecture, device-specific drivers and SoC registers were segregated into a separate **micro-os-plus-iii-devices** repository. This model introduced unnecessary cross-repository coupling and circular dependencies during multi-core bringup.

In the new architecture-specific model, **micro-os-plus-iii-devices** is **dissolved**. Its SoC and driver contents are permanently folded into their owning architecture repositories:

micro-os-plus-iii/	(Kernel, portable POSIX I/O, core test framework)
micro-os-plus-iii-cortexm/	(ARMv7-M & ARMv8-M Cortex-M architectures + STM32 / RP2350 SoCs)
micro-os-plus-iii-aarch32/	(ARMv7-A 32-bit architecture + BCM2837 / Raspberry Pi SoCs)
micro-os-plus-iii-aarch64/	(ARMv8-A 64-bit architecture + BCM2837 / Raspberry Pi SoCs)
micro-os-plus-iii-posix-arch/	(Host POSIX architecture + native uniprocessor & native-smp ports)

Compatibility shims (**micro-os-plus::devices** CMake ALIAS targets) ensure that existing external projects referencing the legacy target continue to configure and link without modification.

Crucial Architecture Decision: Following the dissolution of its contents into the respective architecture repositories, the `micro-os-plus-iii-devices` Git repository and folder are **permanently deleted and eliminated** from the active workspace. The final ecosystem consists strictly of the five primary architecture and kernel repositories.

2. Repository Architecture & Workspace Bootstrap Procedure

2.1 Workspace Directory Layout

Rebuilding the SMP branch cleanly requires an isolated workspace folder (`$WORK`), distinct from the reference workspace in `~/Work2/micro-os-plus/`.

```
# Recommended workspace setup
export WORK="/home/dan/Work2/micro-os-plus-rebuild"
mkdir -p "$WORK" && cd "$WORK"
```

The workspace contains the five primary active repositories, plus the legacy `devices` repository used strictly during Phase 0:

Directory	Repository Name	Initial Branch	Role in Rebuild
<code>\$WORK/micro-os-plus-iii</code>	<code>micro-os-plus-iii</code>	<code>xpack-development</code>	Kernel, schedulers, POSIX I/O, test suites
<code>\$WORK/micro-os-plus-iii-cortexm</code>	<code>micro-os-plus-iii-cortexm</code>	<code>xpack-development</code>	Cortex-M0/M3/M4/M7 baseline + M33/RP2350 SMP
<code>\$WORK/micro-os-plus-iii-posix-arch</code>	<code>micro-os-plus-iii-posix-arch</code>	<code>xpack-development</code>	Host POSIX uniprocessor + dual-core <code>native-smp</code>
<code>\$WORK/micro-os-plus-iii-aarch32</code>	<code>micro-os-plus-iii-aarch32</code>	<code>xpack-development</code>	ARMv7-A 32-bit Cortex-A multi-core port
<code>\$WORK/micro-os-plus-iii-aarch64</code>	<code>micro-os-plus-iii-aarch64</code>	<code>xpack-development</code>	ARMv8-A 64-bit Cortex-A multi-core port
<code>\$WORK/micro-os-plus-iii-devices</code>	<code>micro-os-plus-iii-devices</code>	<code>smp</code> / <code>master</code>	Temporary source for Phase 0 subtree migration (permanently deleted after dissolution)

2.2 Cloning Repositories & Branch Topology

All repositories are cloned from their canonical remotes. The clean baseline is checked out, and a dedicated rebuild branch (named `smp-rebuild` or `smp`) is created:

```
cd "$WORK"

# 1. Clone the core repositories
git clone https://github.com/micro-os-plus/micro-os-plus-iii.git
git clone https://github.com/micro-os-plus/micro-os-plus-iii-cortexm.git
git clone https://github.com/micro-os-plus/micro-os-plus-iii-posix-arch.git
git clone https://github.com/micro-os-plus/micro-os-plus-iii-aarch32.git
git clone https://github.com/micro-os-plus/micro-os-plus-iii-aarch64.git
git clone https://github.com/micro-os-plus/micro-os-plus-iii-devices.git
```

```
# 2. Add local reference remotes pointing to golden source in ~/Work2/micro-os-plus/
for repo in micro-os-plus-iii micro-os-plus-iii-cortexm micro-os-plus-iii-posix-arch \
    micro-os-plus-iii-aarch32 micro-os-plus-iii-aarch64; do
    cd "$WORK/$repo"
    git remote add golden "/home/dan/Work2/micro-os-plus/$repo"
    git fetch golden
    git checkout -b smp origin/xpack-development
done
```

2.3 Toolchain Verification (`check-env.sh`)

Before applying changes, verify that the environment has all required native and cross-compilers, debuggers, QEMU emulators, and build utilities:

```
cd "$WORK/micro-os-plus-iii"
# Execute environment checker
bash scripts/smp/check-env.sh
```

Required toolchain components: - **Build Tools:** CMake `$\ge 3.20$`, Ninja `$\ge 1.10$`, xPacks Package Manager (`xpm`) `$\ge 0.14$`. - **Host Compilers:** GCC 11, 12, 13, or 14; Clang 16, 17, 18, or 19. - **Cross Toolchains:** `arm-none-eabi-gcc` `$\ge 12.3$`, `aarch64-none-elf-gcc` `$\ge 12.3$`. - **Emulators:** `qemu-system-arm` `$\ge 7.0$` (supporting `mps2-an521`, `raspi3b`), `qemu-system-aarch64` `$\ge 7.0$`. - **Utilities:** `unifdef` (for single-core projection verification), Python 3 with `pypdf`, `weasyprint`, and `markdown`.

2.4 Development Linking Protocol (`xpm link`)

In the μ OS++ development workflow, repositories locate each other via `xpm link`. This ensures that local edits in architecture ports are immediately picked up when compiling kernel test suites:

```
# Register port development links
cd "$WORK/micro-os-plus-iii-posix-arch" && xpm install && xpm link
cd "$WORK/micro-os-plus-iii-cortexm" && xpm install && xpm link
cd "$WORK/micro-os-plus-iii-aarch32" && xpm install && xpm link
cd "$WORK/micro-os-plus-iii-aarch64" && xpm install && xpm link

# Link into the kernel test harness
cd "$WORK/micro-os-plus-iii"
xpm link @micro-os-plus/cortexm
xpm link @micro-os-plus/posix-arch
```

2.5 The Test Framework Paradigm of `micro-os-plus-iii`

The test paradigm in μ OS++ IIIe is strictly structured: 1. **The 72-Test Baseline Matrix:** Comprises 24 distinct build configurations (combining native Linux GCC/Clang with Cortex-M0, M3, M4, M7 cross-compilations) each executing three standardized suites: - `rtos-apis-test`: Comprehensive functional validation of all POSIX and RTOS primitives. - `mutex-stress-test`: High-contention lock/unlock concurrency verification. - `cmsis-os-validator-test`: ARM official CMSIS-RTOS validation suite (60/60 test cases). - **Baseline Invariant:** At every *commit* in Phase 1, `xpm run test-all` must produce 72/72 PASS (100% green). 2. **The Multi-Core SMP Test Matrix (`test-smp-all`):** - `test-native-smp`: Real dual-core host execution using pthreads-as-cores and signal-based IPIs. - `test-2xcortex-m33-cmake`: Dual-core ARMv8-M execution on QEMU `mps2-an521`. - `test-aarch32-rpi3b`: 4-core ARMv7-A execution on QEMU `raspi3b`. - `test-aarch64-rpi3b`: 4-core ARMv8-A execution on QEMU `raspi3b`. - `fp-switch`: Floating-point register corruption stress test under aggressive preemption.

3. Phase 0: Pre-Migration Subtree Dissolution of Legacy

`devices`

3.1 Why the `devices` Repository Must Disappear

The legacy `micro-os-plus-iii-devices` repository isolated silicon device drivers (`soc/stm32f4xx`, `soc/rp2350`, `soc/bcm2837`) outside their corresponding CPU architecture repositories. This caused: - Split architectural headers: Cortex-M33 register definitions lived in `cortexm`, but RP2350 SIO spinlocks lived in `devices`. - Multi-repository synchronization locks: Modifying an interrupt controller required coordinated commits across two repositories before tests could build. - Divergence from upstream xPack distribution packaging.

By folding device support directly into `cortexm`, `posix-arch`, `aarch32`, and `aarch64`, each architecture repository becomes a complete, self-contained hardware abstraction layer.

3.2 History-Preserving `git subtree` Extraction Protocol

Dissolution must **not** be performed via flat file copying; commit history must be 100% preserved. This is achieved using `git subtree add`:

```
cd "$WORK/micro-os-plus-iii-cortexm"

# Subtree add Cortex-M silicon devices from devices repo
git subtree add --prefix=devices "$WORK/micro-os-plus-iii-devices" master

# Restructure into standard architecture directories
mv devices/soc/stm32f4xx soc/stm32f4xx
mv devices/soc/rp2350 soc/rp2350
mv devices/drivers drivers
rm -rf devices

git add soc drivers
git commit -m "refactor(devices): fold Cortex-M SoC and drivers subtree into cortexm port"
```

The identical procedure is executed for `posix-arch` (folding `soc/native`), `aarch32` (folding `soc/bcm2837`), and `aarch64`.

3.3 CMake Target Realignment & Removal of `UOS_DEVICES_DIR`

Prior to dissolution, both `posix-arch` and `cortexm` expected an external sibling `micro-os-plus-iii-devices` directory and contained the following legacy boilerplate in their root `CMakeLists.txt`:

```

set (UOS_DEVICES_DIR "${uos_siblings}/micro-os-plus-iii-devices"
    CACHE PATH "µOS++ III devices working copy")

foreach (_dep_var IN ITEMS UOS_SMP_DIR UOS_DEVICES_DIR)
    ...
    if (NOT EXISTS "${${_dep_var}}/CMakeLists.txt")
        message (FATAL_ERROR "Cannot find the ${_repo} repository at ${${_dep_var}}...")
    endif ()
endforeach ()

add_subdirectory ("${UOS_DEVICES_DIR}" micro-os-plus-iii-devices)

```

If left unchanged after `micro-os-plus-iii-devices` is deleted, any subsequent CMake configure or test command fails immediately with:

```

CMake Error at /tmp/micro-os-plus-iii-posix-arch/CMakeLists.txt:

  Cannot find the devices repository at /tmp/micro-os-plus-iii-devices.

```

Realignment in `micro-os-plus-iii-posix-arch/CMakeLists.txt`

1. Remove `UOS_DEVICES_DIR` and its validation loop from the dependency checks.
2. Define the local block device and filesystem targets directly using the absorbed `drivers/` and `soc/native/` files:

```

# Dissolved devices support: hostfile block device and flatfs
if (NOT TARGET micro-os-plus::devices-hostfile)
    add_library (micro-os-plus-iii-devices-hostfile-interface INTERFACE)
    target_include_directories (
        micro-os-plus-iii-devices-hostfile-interface
        INTERFACE
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/include"
            "${CMAKE_CURRENT_SOURCE_DIR}/soc/native/include"
    )
    target_compile_definitions (
        micro-os-plus-iii-devices-hostfile-interface INTERFACE SD_BACKEND_HOSTFILE
    )
    target_sources (
        micro-os-plus-iii-devices-hostfile-interface
        INTERFACE
            "${CMAKE_CURRENT_SOURCE_DIR}/soc/native/src/sd_hostfile.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/src/flatfs.cpp"
    )
    add_library (micro-os-plus::devices-hostfile ALIAS
        micro-os-plus-iii-devices-hostfile-interface)
endif ()

```

```

# Compatibility alias for full devices suite (FatFs + block device)
if (NOT TARGET micro-os-plus::devices)
    add_library (micro-os-plus-iii-devices-interface INTERFACE)
    target_include_directories (
        micro-os-plus-iii-devices-interface
        INTERFACE
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/include"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/fatfs"
    )
    target_sources (
        micro-os-plus-iii-devices-interface
        INTERFACE
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/src/sd.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/src/flatfs.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/src/usb_dwc2.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/fatfs/ff.c"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/fatfs/diskio_sd.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/fatfs/fatfs_hw.cpp"
    )
    add_library (micro-os-plus::devices ALIAS micro-os-plus-iii-devices-interface)
endif ()

```

Realignment in `micro-os-plus-iii-cortexm/CMakeLists.txt`

1. Remove `UOS_DEVICES_DIR` and its validation loop from the dependency checks.
2. Define the local STM32F4xx and RP2350 silicon support targets directly using the absorbed `soc/` files:

```

# Dissolved SoC targets: STM32F4xx and RP2350
if (NOT TARGET micro-os-plus::soc-stm32f411xe)
    add_library (micro-os-plus-iii-soc-stm32f4xx-common-interface INTERFACE)
    target_include_directories (
        micro-os-plus-iii-soc-stm32f4xx-common-interface
        INTERFACE
            "${CMAKE_CURRENT_SOURCE_DIR}/soc/stm32f4xx/include"
            "${CMAKE_CURRENT_SOURCE_DIR}/soc/stm32f4xx/include/cmsis"
    )
    target_sources (
        micro-os-plus-iii-soc-stm32f4xx-common-interface
        INTERFACE "${CMAKE_CURRENT_SOURCE_DIR}/soc/stm32f4xx/src/system_stm32f4xx.c"
    )

    add_library (micro-os-plus-iii-soc-stm32f411xe-interface INTERFACE)
    target_link_libraries (

```

```

    micro-os-plus-iii-soc-stm32f411xe-interface
    INTERFACE micro-os-plus-iii-soc-stm32f4xx-common-interface
)
target_sources (
    micro-os-plus-iii-soc-stm32f411xe-interface
    INTERFACE "${CMAKE_CURRENT_SOURCE_DIR}/soc/stm32f4xx/src/vectors_stm32f411xe.c"
)
add_library (micro-os-plus::soc-stm32f411xe
    ALIAS micro-os-plus-iii-soc-stm32f411xe-interface)

add_library (micro-os-plus-iii-soc-stm32f412rx-interface INTERFACE)
target_link_libraries (
    micro-os-plus-iii-soc-stm32f412rx-interface
    INTERFACE micro-os-plus-iii-soc-stm32f4xx-common-interface
)
target_sources (
    micro-os-plus-iii-soc-stm32f412rx-interface
    INTERFACE "${CMAKE_CURRENT_SOURCE_DIR}/soc/stm32f4xx/src/vectors_stm32f412rx.c"
)
add_library (micro-os-plus::soc-stm32f412rx
    ALIAS micro-os-plus-iii-soc-stm32f412rx-interface)
endif ()

if (NOT TARGET micro-os-plus::soc-rp2350)
    add_library (micro-os-plus-iii-soc-rp2350-interface INTERFACE)
    target_include_directories (
        micro-os-plus-iii-soc-rp2350-interface
        INTERFACE
            "${CMAKE_CURRENT_SOURCE_DIR}/soc/rp2350/include"
            "${CMAKE_CURRENT_SOURCE_DIR}/soc/rp2350/include/cmsis"
    )
    target_sources (
        micro-os-plus-iii-soc-rp2350-interface
        INTERFACE "${CMAKE_CURRENT_SOURCE_DIR}/soc/rp2350/src/system_rp2350.c"
    )
    add_library (micro-os-plus::soc-rp2350
        ALIAS micro-os-plus-iii-soc-rp2350-interface)
endif ()

```

Realignment in `aarch32` and `aarch64`

Both `micro-os-plus-iii-aarch32/CMakeLists.txt` and `micro-os-plus-iii-aarch64/CMakeLists.txt` compile the dissolved `soc/bcm2837` sources locally into `micro-os-plus::aarch32` and `micro-os-plus::aarch64` interface targets, with zero external dependency on `devices`.

Furthermore, following the dissolution of `micro-os-plus-iii-devices`, tests exercising the SD host (`sd_test`, `smp-mat-sdcard-test`), USB controller (`usb_test`), and file systems require the `micro-os-plus::devices` target. Both AArch32 and AArch64 declare this target locally from their absorbed `drivers/` and `soc/` trees:

```
# Dissolved devices support: SD host, USB, and FatFs stack
if (NOT TARGET micro-os-plus::devices)
    add_library (micro-os-plus-iii-devices-interface INTERFACE)
    target_include_directories (
        micro-os-plus-iii-devices-interface
        INTERFACE
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/include"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/fatfs"
    )
    target_sources (
        micro-os-plus-iii-devices-interface
        INTERFACE
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/src/sd.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/src/flatfs.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/src/usb_dwc2.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/fatfs/ff.c"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/fatfs/diskio_sd.cpp"
            "${CMAKE_CURRENT_SOURCE_DIR}/drivers/fatfs/fatfs_hw.cpp"
    )
    add_library (micro-os-plus::devices ALIAS micro-os-plus-iii-devices-interface)
endif ()
```

Duplicate Port Target Prevention in Platform Harness

In the μ OS++ test framework (`tests/cmake/tests-main.cmake`), the architecture port under test is added directly via `add_subdirectory ("${UOS_<PORT>_DIR}" "port-bin")`. Therefore: - The architecture port (`micro-os-plus-iii-aarch32` or `micro-os-plus-iii-aarch64`) must **never** be listed in `dependencies-folders.cmake` (`xpack_dependencies_folders`). - Listing the port in `dependencies-folders.cmake` causes CMake policy `CMP0002` violations (`add_library cannot create target ... because another target with the same name already exists`). - Platforms list only test suite directories and portable xPacks (`mutex-stress`, `rtos-apis`, `arm-cmsis-rtos-validator`, `chan-fatfs`) in `dependencies-folders.cmake`.

Shared Test Support Scaffolding Realignment (`test_smpl` $\rightarrow$ `tests/smp-support`)

When the shared test scaffolding was consolidated into `tests/smp-support/` (`hw_result.hpp` and `board-contract.cpp`): - `micro-os-plus::test-support` in `micro-os-plus-iii/CMakeLists.txt` exposes `${CMAKE_CURRENT_SOURCE_DIR}/tests/smp-support/include` (with a fallback to `test_smpl/include`). - A backwards-compatibility symlink `test_smpl -> tests/smp-support` is maintained in the

root of `micro-os-plus-iii` so that legacy test runners (`run-hw.sh`, `run-host.sh`, `run-qemu.sh`) and scripts referencing `$SMP_DIR/test_smpl/` execute without error.

3.4 Phase 0 Verification Gate

```
# Verify all existing uniprocessor boards configure and build cleanly
cd "$WORK/micro-os-plus-iii"
xpm run test-all # Must remain 72/72 PASS
```

3.5 Permanent Deletion of the Legacy `devices` Repository and Working Directory

Once the subtree extraction into `cortexm`, `posix-arch`, `aarch32`, and `aarch64` is complete and verified, the `micro-os-plus-iii-devices` repository and working directory have fulfilled their transitional purpose.

The `micro-os-plus-iii-devices` Git clone/directory must be **permanently deleted from the active workspace**:

```
# Permanently delete the dissolved devices repository clone from the workspace
rm -rf "$WORK/micro-os-plus-iii-devices"
```

Key Architectural Rules Post-Deletion:

- 1. Zero External Device Dependency:** No architecture repository or kernel build configuration may reference or depend on an external `micro-os-plus-iii-devices` directory.
- 2. Self-Contained Ports:** Silicon drivers (`soc/*`) and neutral drivers (`drivers/*`) are compiled and maintained directly inside each architecture repository (`cortexm`, `posix-arch`, `aarch32`, `aarch64`).
- 3. Retired Repository:** The standalone `micro-os-plus-iii-devices` repository is officially retired and archived; it is never propagated into future `xpack-development` or `smp` deployment cycles.

4. Phase 1: Single-Core Defect Corrections & Hardening (PR #1 through PR #22)

Phase 1 isolates all bugfixes, standard conformance corrections, and allocator hardening that are **pure uniprocessor improvements**. Landing these 22 commits first guarantees that single-core $\mu\text{OS++}$ is completely defect-free before any multi-core scheduling logic is introduced.

Summary Table of Phase 1 Pull Requests

PR #	Subject / Theme	File(s) Touched	Problem & Failure Mode Averted	Verification Gate
#1	POSIX <code>dirent.h</code> ISO C Conformance	<code>include/cmsis-plus/posix/dirent.h</code>	Empty struct <code>typedef struct { ; } DIR;</code> violates ISO C99/C11 §6.7.2.1; triggers <code>-Wextra-semi</code> .	72/72 pass; clean <code>-Wextra-semi</code>
#2	List Iterator Access Semantics	<code>include/cmsis-plus/utils/lists.h</code>	<code>node_>next</code> accessed as data member instead of method <code>node_>next()</code> , breaking C++17/20 concepts.	72/72 pass; template concepts clean
#3	Newlib Syscall Return Types	<code>posix-io/c-syscalls-aliases-standard.h</code>	Newlib 64-bit uses <code>_READ_WRITE_RETURN_TYPE (int or _ssize_t)</code> , causing conflicting weak alias signatures.	72/72 pass; reentrant tests clean
#4	Export Thread Suspend Symbol	<code>src/rtos/os-thread.cpp</code>	Out-of-line <code>inline void suspend()</code> prevented symbol emission; undefined reference in C API wrapper.	72/72 pass; wrapper links cleanly
#5	Memory Allocator Usable Size	<code>os-memory.*</code> , <code>malloc.cpp</code> , <code>first-fit-top.*</code>	Missing <code>do_usable_size()</code> broke <code>realloc()</code> ; <code>calloc()</code> lacked multiplication overflow checks.	72/72 pass; usable size verified
#6	C++17 Aligned Allocation	<code>src/libcpp/new.cpp</code>	Missing standard <code>operator new/delete(align_val_t)</code> overloads for over-aligned types.	72/72 pass; aligned new clean
#7	Static Error Category Storage	<code>src/libcpp/system-error.cpp</code>	Temporary category objects left dangling pointers inside <code>throw std::system_error</code> .	72/72 pass; static lifetime clean
#8	Chrono 64-Bit Duration Overflow	<code>src/libcpp/chrono.cpp</code>	<code>cycles * 1e9 / freq</code> overflowed 64-bit int after ~13 minutes. Decompose into seconds and remainder.	72/72 pass; chrono math overflow-free
#9	CMSIS-RTOS C Wrapper Cleanups	<code>src/rtos/os-c-wrapper.cpp</code>	Default timer must be one-shot; polymorphic mutex delete; cast <code>millisec</code> to	72/72 pass; validator 60/60 clean

PR #	Subject / Theme	File(s) Touched	Problem & Failure Mode Averted	Verification Gate
			<code>uint64_t</code> prior to <code>*</code> <code>1000</code> .	
#10	POSIX I/O FD Table Mutexing	<code>src/posix-io/file-descriptors-manager.cpp</code>	Unprotected descriptor table corrupted under concurrent multi-threaded <code>open()</code> / <code>close()</code> .	72/72 pass; concurrent I/O stress pass
#11	POSIX I/O Deferred List Locking	<code>posix-io/file-system.h</code> , <code>net-stack.h</code>	Concurrent recycling of deferred file/socket nodes caused corrupted intrusive list links.	72/72 pass; list recycling clean
#12	Block Device Sector Size Check	<code>src/posix-io/block-device.cpp</code>	Inverted check in <code>vfcntl</code> : tested <code>!= 0</code> instead of <code>== 0</code> , falsely rejecting valid block devices.	72/72 pass; valid block size returned
#13	ARMv8-M Semihosting Traps	<code>include/cmsis-plus/arm-semihosting.h</code>	Missing <code>__ARM_ARCH_8M_MAIN</code> macro guards and <code>SEMIHOST_TRAP_HLT</code> support.	72/72 pass; ARM semihosting pass
#14	ARMv8-M Exception Handlers	<code>src/startup/exception-handlers.c</code>	SCB register setup and fault dump routines lacked <code>__ARM_ARCH_8M_MAIN</code> guards.	72/72 pass; fault handlers compile
#15	Semihosting <code>fstat()</code> & Mirror	<code>src/semihosting/c-syscalls-semihosting.cpp</code>	Assign <code>S_IFCHR</code> only when mode unset; add weak <code>os_board_console_merror()</code> hook.	72/72 pass; console mirror clean
#16	Timer Callback Decoupling	<code>internal/os-lists.cpp</code> , <code>os-timer.cpp</code>	Executing timer callbacks inside critical section caused deadlocks; drift catch-up guard.	72/72 pass; timer callback mutexes clean
#17	Mutex Priority Ceiling Protocol	<code>src/rtos/os-mutex.cpp</code>	Enforce ceiling before acquiring ownership; track maximum boost among multiple waiters.	72/72 pass; <code>mutex-stress-test</code> pass
#18	Thread Lifecycle State & Reaper	<code>os-c-decls.h</code> , <code>os-thread.h</code> , <code>os-idle.cpp</code>	Add <code>state::destroying = 7</code> ; idle thread reaper safely validates no core is running thread before free.	72/72 pass; thread reaper green
#19			Atomic link node & mutex unlock under	

PR #	Subject / Theme	File(s) Touched	Problem & Failure Mode Averted	Verification Gate
	CondVar Lost-Signal Elimination	<code>src/rtos/os-condvar.cpp</code> , <code>os-condvar.h</code>	<code>scheduler::critical_section</code> ; bind condvar to clock.	72/72 pass; condvar concurrency green
#20	<code>std::thread</code> Handle Sync	<code>thread-cpp.h</code> , <code>thread_internal.h</code>	<code>thread::join()</code> waits on native thread handle; retain functor object to eliminate use-after-free.	72/72 pass; functor lifetime verified
#21	MQueue Preemptive Reschedule	<code>src/rtos/os-mqueue.cpp</code>	Call <code>port::scheduler::reschedule()</code> immediately after send/receive wakeups.	72/72 pass; queue preemption green
#22	High-Res Clock Port Sync	<code>os-decls.h</code> , <code>os-clocks.cpp</code> , Port inlines	Declares <code>has_hardware_counter()</code> ; fast-paths now(); SysTick ICSR pending overflow inspection.	72/72 pass; ICSR pending fix verified

5. Phase 2: Common Multi-Core SMP Kernel Infrastructure & Host Runtime (PR #23 / Step 14)

5.1 Architectural Foundations of the SMP Kernel

Phase 2 constitutes the architectural core of the entire SMP migration. It transitions the kernel from a single-CPU RTOS to a Symmetric Multiprocessing engine capable of executing threads concurrently across N physical cores.

Key data structure transformations in `include/cmsis-plus/rtos/os-sched.h` and `src/rtos/os-core.cpp`:

- **Per-CPU Current Threads**: Replaces scalar `current_thread_` with an array indexed by `port_cpu_id()`:

```
cpp #if defined(OS_USE_SMP_SCHEDULER) extern thread* volatile current_thread_[OS_NCPU]; #else extern thread* volatile current_thread_; #endif
```
- **Per-CPU Ready Lists**: Replaces the monolithic ready list with `ready_list_[OS_NCPU]`, eliminating global queue lock bottlenecking during normal thread execution.
- **Recursive Kernel Spinlock** (`_smp_klock`): Protects scheduler state transitions. Backed by architecture hardware spinlocks (e.g. RP2350 SIO spinlock 0, Cortex-M33 LDREX/STREX, or POSIX mutex).

5.2 The 5-Stage Stack Pointer Claim/Publish Protocol

To prevent fatal stack corruption when two cores attempt to switch to or from the same thread simultaneously, the scheduler implements a strict 5-stage handshake:

1. **Claim**: The incoming thread stack pointer is atomically set to `nullptr` on the target core, marking it actively running.
2. **Spill**: The outgoing thread's callee-saved registers are pushed to its current stack.
3. **Switch**: Hardware Stack Pointer register (SP) is updated to the incoming thread's stack.
4. **Publish**: The outgoing thread's saved stack pointer is published to `from->context_.port_.stack_ptr = sp` **only after** state is completely saved.
5. **Restore**: Callee-saved registers are popped from the new stack, and thread execution resumes.

5.3 Prohibiting Context Switch While Holding Spinlock

A primary invariant in $\mu\text{OS++}$ SMP: **A hardware context switch must never be executed while holding `_smp_klock`**. Context switching while holding the kernel spinlock causes deadlocks across cores. Instead, `internal_switch_threads()` schedules the switch, marks the CPU as requiring rescheduling, releases `_smp_klock`, and allows the architecture PendSV interrupt or signal handler to execute the actual register swap.

5.4 Phase 2 Verification Gates

1. The `unifdef` **Single-Core Invariant**: Running `unifdef -UOS_USE_SMP_SCHEDULER` on the Phase 2 kernel source must produce a tree that is **100% byte-identical to Step 22**. Uniprocessor builds incur zero code bloat and zero overhead.
2. **72/72 Uniprocessor Test Gate**: `xpm run test-all` passes with 100% green status.

3. **Genuine Dual-Core Host Verification (`native-smp`):** Executes `tests/platforms/native-smp/` under Linux host using 2 OS threads as virtual CPU cores: `bash cd "$WORK/micro-os-plus-iii/tests" xpm run test-native-smp # Must execute and pass: # 1. rtos-apis-test (dual-core concurrency) # 2. mutex-stress-test (contention across cores) # 3. cmsis-os-validator-test (60/60 test cases)`

6. Phase 3: Semantic Versioning, Port Releases & Silicon Extensions (PR #24 through PR #27)

6.1 PR #24: Port Package Semantic Releases

Publishes the formal multi-core hardware abstraction contract: - `micro-os-plus-iii-posix-arch`: Version bumped from `1.0.1` $\rightarrow$ `1.1.0`. - `micro-os-plus-iii-cortexm`: Version bumped from `1.1.0` $\rightarrow$ `1.2.0`. Local semantic git tags (`v1.1.0`, `v1.2.0`) are generated for repeatable downstream dependency resolution.

6.2 PR #25: Execution of Devices Subtree Dissolution, CMake Realignment & Repository Deletion

Finalizes the historical merge of `soc/*` and `drivers/*` into the owning architecture ports, establishing clean build targets and permanent ALIAS compatibility shims.

Key Implementation Actions:

- Subtree Merge:** Folds `soc/stm32f4xx` and `soc/rp2350` into `cortexm`, `soc/native` and `drivers/*` into `posix-arch`, and `soc/bcm2837` into `aarch32` / `aarch64`.
- CMakeLists.txt Realignment:** - In `posix-arch/CMakeLists.txt`: Remove `UOS_DEVICES_DIR` dependency check. Declare `micro-os-plus::devices-hostfile` (from `soc/native/src/sd_hostfile.cpp` and `drivers/src/flatfs.cpp` with `SD_BACKEND_HOSTFILE`) and `micro-os-plus::devices` directly. - In `cortexm/CMakeLists.txt`: Remove `UOS_DEVICES_DIR` dependency check. Declare `micro-os-plus::soc-stm32f411xe`, `micro-os-plus::soc-stm32f412rx`, and `micro-os-plus::soc-rp2350` directly from the local `soc/` paths.
- Repository Deletion:** Immediately upon completion of the subtree graft and CMake realignment, the `micro-os-plus-iii-devices` repository folder and Git repository are **permanently deleted from the workspace**, retiring the standalone repository and guaranteeing that all downstream projects build and test 100% self-contained.

6.3 PR #26: Modern Multi-Core Silicon Ports (Cortex-M33 & RP2350)

Introduces genuine silicon hardware multi-core implementations: - **ARM Cortex-M33 Port:** Supports ARMv8-M architecture, Message Handling Unit (MHU) Inter-Processor Interrupts (IPI), and per-core SysTick clocks. - **Raspberry Pi RP2350 Port:** Implements recursive `_smp_klock` backed by SIO Hardware Spinlock 0 (`*(volatile uint32_t*)(0xD0000100)`), Core 1 wake-up mailbox, and FIFO IRQ 25. - **Kernel IPI Dispatching:** In `src/rtos/os-thread.cpp`, dispatches wake-up IPIs whenever a thread unblocks on an alternate core (`OS_INTEGER_RTOS_PORT_NCPU > 1`).

6.4 PR #27: Modular Add-Only CMake Build Architecture

Splits the monolithic CMake interface into clean, granular sub-targets: - `micro-os-plus::iii-core`: Bare kernel RTOS engine without POSIX I/O. - `micro-os-plus::iii-posix-io`: File system, mount tables, and character devices. - `micro-os-plus::port-smp-decls`: Pure SMP forward declarations for thin ports. - Adds cross-compilation toolchain profiles (`cmake/toolchains/arm-none-eabi.cmake`, `cmake/`

```
toolchains/aarch64-none-elf.cmake ). - Adds QEMU 2-core platform
tests/platforms/2xcortex-m33/ ( mps2-an521 --cpu cortex-m33 --smp 2 ).
```

7. Phase 4: Multi-Core Test Suites, Multi-Arch Platforms & CI Harness (PR #28 & PR #29)

7.1 PR #28: Multi-Core FPU Stress Suite & Scaffolding Relocation

- **FPU Context Switch Stress Suite** (`tests/sources/fp-switch/`): 6 threads computing mathematical floating-point series across cores under aggressive 1ms preemptive slicing (3000 ticks). Detects missing hardware FPU lazy-stacking or register corruptions.
- **Scaffolding Relocation**: Moves legacy root `test_smp/` into `tests/smp-support/`, cleanly exported as `micro-os-plus::test-support` providing standardized test reporting and hardware exit registers.

7.2 PR #29: Multi-Architecture 4-Core Platforms & Composite CI Action

- **AArch32 & AArch64 4-Core Platforms**: Implements QEMU `raspi3b` platforms (`tests/platforms/aarch32-rpi3b/`, `tests/platforms/aarch64-rpi3b/`) executing across 4 ARM Cortex-A53 cores.
 - **Composite Test Action** (`test-smp-all`): Registered in `tests/package.json`, executing the complete multi-core test matrix in one command:

```
json "test-smp-all": { "description": "Run all SMP test suites across Native, Cortex-M33, and AArch32/64", "actions": [ "xpm run test-native-smp", "xpm run test-2xcortex-m33-cmake", "xpm run test-aarch32-rpi3b", "xpm run test-aarch64-rpi3b" ] }
```
-

8. Phase 5: Documentation Suite, Master Reconciliation & Silicon Wiring (PR #30 through PR #32)

8.1 PR #30: Comprehensive Documentation & Specification Runbooks Suite

Integrates all architectural runbooks and migration specifications into `docs/render-pdfs.sh`. Builds the complete 16-document PDF publication suite.

8.2 PR #31: Final Master Reconciliation & Repository Cleansing

- Merges upstream `origin/xpack-development` cleanly.
- Restores `.github/workflows/ci.yml`, `README.md`, `LICENSE`, and Doxygen templates to be **100% byte-identical** to upstream baseline.
- Purges temporary transition scripts (`scripts/smp/`).

8.3 PR #32: Dedicated Raspberry Pi Pico 2 Physical Hardware Platform Wiring

Adds standalone hardware platform `tests/platforms/cortexm-pico2/` enabling direct OpenOCD SWD flashing and UART verification on physical RP2350 silicon. Implements Core 1 secondary boot handshake via SIO mailbox registers (`sio_hw->fifo_wr` / `sio_hw->fifo_rd`).

9. Operator's Execution Manual: The Step-by-Step Lifting Loop

9.1 The Golden 4-Command Development Cycle per Step

For each of the 32 pull requests, the developer executes a standardized four-command loop:

```
+-----+ +-----+ +-----+ +-----+
| 1. NEW STEP | ---> | 2. APPLY CHUNK | ---> | 3. VERIFY GATE | ---> | 4. COMMIT / ADV |
| Fork step branch | | Lift clean diff | | Compile & Test | | Atomic git commit|
+-----+ +-----+ +-----+ +-----+
```

Step 1: Fork the Step Branch

```
cd "$WORK/micro-os-plus-iii"

# Creates step/NN branched from step/NN-1 (or xpack-development at Step 1)

bash scripts/smp/new-step.sh <NN>
```

Step 2: Apply the Step's Isolated Chunk

```
# Sourced directly from golden reference repo in ~/Work2/micro-os-plus/

bash scripts/smp/chunks/step<NN>.sh

# Or inspect unified diff:

bash scripts/smp/show-chunk.sh <NN> <file...>
```

Step 3: Execute the Verification Gate

```
# Validates pristine rules, unifdef projection, and runs 72/72 test matrix

bash scripts/smp/verify-step.sh <NN>
```

Step 4: Commit and Advance

```
# Creates atomic git commit with conventional commit header

bash scripts/smp/advance-step.sh <NN> "commit message description"
```

9.2 Stacking Pull Requests on GitHub

When submitting the 32 steps to GitHub: 1. Push each local step branch: `git push -u origin step/<NN>`. 2. Open Pull Request on GitHub: - For PR #1: Target base `xpack-development` $\rightarrow$ Head `step/01`. - For PR #2: Target base `step/01` $\rightarrow$ Head `step/02`. - For PR #N: Target base

`step/<N-1>` $\$ \leftarrow$ Head `step/<N>`. 3. When upstream merges PR #1 into `xpack-development`, use GitHub's web UI to retarget PR #2's base from `step/01` to `xpack-development`.

9.3 Cherry-Picking Commits Back into `xpack-development`

Because each commit is strictly self-contained and touches only files relevant to its single theme, any individual commit or group of commits can be cherry-picked directly into `xpack-development`:

```
cd "$WORK/micro-os-plus-iii"
git checkout xpack-development
git fetch origin

# Example: Cherry-pick PR #1 (dirent) and PR #2 (lists) directly
git cherry-pick <PR_1_COMMIT_HASH>
git cherry-pick <PR_2_COMMIT_HASH>

# Verify baseline tests remain 100% green
xpm run test-all
```

10. Comprehensive Verification Matrix & Acceptance Gates

Every pull request satisfies an explicit, automated acceptance gate prior to being advanced:

PR ID	Theme & Domain	Repositories	Self-Contained?	Automated Gate Command	Acceptance Pass Criteria
PR #1	POSIX <code>dirent.h</code> Conformance	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; <code>-Wextra-semi</code> clean
PR #2	List Iterator Member Semantics	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; template concepts clean
PR #3	Newlib Syscall Return Types	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; newlib reentrant tests pass
PR #4	Thread Suspend Symbol Export	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; C wrapper resolves symbol
PR #5	Memory Allocator Usable Size	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; <code>malloc_usable_size</code> verified
PR #6	C++17 Aligned Allocation	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; aligned <code>operator new</code> clean
PR #7	Static Error Category Storage	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; static storage eliminates dangling ptrs
PR #8	Steady Clock Duration Overflow	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; chrono math overflow-free
PR #9	CMSIS-RTOS C Wrapper Cleanups	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; validator (60/60) clean
PR #10	POSIX I/O FD Table Mutex	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; concurrent I/O stress green
PR #11	POSIX I/O Deferred List Locking	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; list recycling thread-safe
PR #12	Block Device Sector Size Check	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; valid block size returned
PR #13	ARMv8-M Semihosting Traps	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; ARM semihosting tests pass
PR #14	ARMv8-M Exception Handlers	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; ARMv8-M fault vectors compile
PR #15	Semihosting <code>fstat()</code> & Mirror	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; mode preserved &

PR ID	Theme & Domain	Repositories	Self-Contained?	Automated Gate Command	Acceptance Pass Criteria
					console mirror hook
PR #16	Software Timer Decoupling	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; timer callback mutexes clean
PR #17	Mutex Priority Ceiling	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; <code>mutex-stress-test</code> passes
PR #18	Thread Lifecycle & Reaper	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; dynamic thread reaper green
PR #19	CondVar Lost-Signal Elimination	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; condvar concurrency stress green
PR #20	<code>std::thread</code> Handle Sync	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; functor closure lifetime verified
PR #21	MQueue Preemptive Reschedule	Kernel	standalone	<code>xpm run test-all</code>	72/72 tests pass; message queue preemption green
PR #22	High-Res Clock Port Sync	K + C + P	atomic	<code>xpm run test-all</code>	72/72 tests pass; ICSR pending fix verified
PR #23	Multi-Core SMP Core & Host	K + C + P	atomic	<code>scripts/smp/verify-step.sh 14</code>	<code>unifdef</code> zero-diff + 72/72 + <code>native-smp</code> (3/3) PASS
PR #24	Port Semantic Releases	P + C	standalone	<code>git tag -l</code>	<code>v1.1.0</code> and <code>v1.2.0</code> local tags generated
PR #25	Devices Subtree Dissolution	P + C + Arch	standalone	<code>xpm run build ...</code>	Subtree merged; CMake ALIAS targets resolve; <code>devices</code> folder deleted from workspace
PR #26	Modern Silicon Ports	C + K	add-only	<code>scripts/smp/verify-step.sh 25</code>	M33/RP2350 port targets compile cleanly
PR #27	Modular Add-Only CMake	K + C	add-only	<code>scripts/smp/verify-step.sh 26</code>	72/72 baseline + <code>2xcortex-m33</code> (4/4) PASS
PR #28	FPU Context Switch Stress	Kernel	add-only	<code>scripts/smp/verify-step.sh 27</code>	<code>fp-switch</code> passes on <code>2xcortex-m33</code> (4/4)

PR ID	Theme & Domain	Repositories	Self-Contained?	Automated Gate Command	Acceptance Pass Criteria
PR #29	Multi-Arch 4-Core Platforms	K + A32 + A64	add-only	<code>scripts/smp/verify-step.sh 28</code>	<code>test-smp-all</code> passes (Native, M33, AArch)
PR #30	Documentation Suite	Kernel	standalone	<code>docs/render-pdfs.sh</code>	All PDFs compile cleanly without warnings
PR #31	Final Merge & Cleansing	All Repos	clean-merge	<code>bash scripts/smp/finalize.sh</code>	Upstream CI metadata 100% byte-identical
PR #32	Pico 2 Hardware Platform	K + C	add-only	<code>xpm run build --config cortexm-pico2</code>	Pico 2 platform compiles for hardware SWD

10.2 Final Parity Verification with Golden Source

At the conclusion of Step 32: - The rebuilt `smp` branch in `$WORK/micro-os-plus-iii` is compared against `/home/dan/Work2/micro-os-plus/micro-os-plus-iii`: `bash diff -ruN -x ".git" "$WORK/micro-os-plus-iii" "/home/dan/Work2/micro-os-plus/micro-os-plus-iii"` - **Acceptance Invariant:** The source code trees across all five repositories (`micro-os-plus-iii`, `cortexm`, `posix-arch`, `aarch32`, `aarch64`) achieve **100% functional and structural parity** with the reference implementation in `~/Work2/micro-os-plus/`, but possess a **clean, granular, 32-commit GitHub Pull Request history** ready for immediate upstream integration.